

ACESSO A DATOS

Semana 1 · Ejercicios de persistencia, arquitectura y repaso SQL

Ejercicio 1. Persistencia

1. No necesita persistencia pues es temporal.
2. Persistente pues te interesa mantener esa lista de clientes.
3. Temporal, pues no es el resultado final.
4. Persistente ya que es importante mantener las notas de los alumnos finales.
5. Temporal pues se usa para hacer otro calculo.
6. Persistente pues quieres saber los pedidos de tu tienda online.

Ejercicio 2. Elegir un mecanismo de almacenamiento

1. En XML/JSON ya que toda la configuración se guarda en JSON.
2. En BBDD ya que es mucha información y necesitas mantenerla, por tanto, necesitas servidores externos para gestionarla.
3. En XML/JSON ya es la mejor forma de intercambiar documentos estructurados entre 2 sistemas.
4. En XML/JSON o fichero de texto, si la información no se puede representar con clave valor sería fichero de texto, en caso contrario XML/JSON.
5. En BBDD pues te interesa guardar la información.
6. En la RAM ya que es temporal.

Ejercicio 3. Modelo relacional

1. 3 registros
2. 4 campos
3. Id
4. Si a no ser que el campo nombre sea unique, pues el único campo que no se puede repetir es el id que es la PK.
5. No, porque es la PK que es única.

Ejercicio 4. Crear una base de datos.

```
DROP DATABASE IF EXISTS biblioteca;  
CREATE DATABASE IF NOT EXISTS biblioteca;  
USE biblioteca;
```

```
CREATE TABLE libros (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  titulo VARCHAR(100) UNIQUE NOT NULL,  
  autor VARCHAR(25),  
  precio FLOAT(2),  
  anio_publicacion INT
```

```
);
```

Ejercicio 5. INSERT

```
INSERT INTO libros (titulo, autor, precio, anio_publicacion) VALUES
("El Quijote", "Luis", 40.53, 2000),
("1984", "Pedro", 33.12, 2001),
("El Hobbit", "Javier", 23.55, 2002),
("Fundación", "Juan", 34.12, 2003),
("Dune", "Manuel", 12.53, 2004),
("La sombra del viento", "Bartolomeo", 84.23, 2005);
```

Ejercicio 6. SELECT básico

```
SELECT * FROM libros;
SELECT titulo, autor FROM libros;
SELECT * FROM libros WHERE precio > 15;
SELECT * FROM libros WHERE anio_publicacion > 2000;
SELECT * FROM libros ORDER BY titulo;
SELECT * FROM libros ORDER BY precio DESC;
```

Ejercicio 7. Filtros

```
SELECT * FROM libros WHERE precio > 10 AND precio < 20; /* SELECT * FROM libros WHERE
precio BETWEEN 10 AND 20; */
SELECT * FROM libros WHERE autor = "Juan";
SELECT titulo FROM libros WHERE titulo COLLATE utf8mb4_bin LIKE 'E1%';
SELECT titulo FROM libros WHERE titulo LIKE "% de %";
SELECT * FROM libros WHERE anio_publicacion < 1990 AND precio < 20;
```

Ejercicio 8. UPDATE

```
UPDATE libros SET precio = 500 WHERE titulo = "1984";
UPDATE libros SET autor = "Marta" WHERE titulo LIKE "%quijote";
UPDATE libros SET precio = precio + 2;
UPDATE libros SET anio_publicacion = 1000 WHERE id = 6;
```

Ejercicio 9. DELETE

```
DELETE FROM libros WHERE id = 5;
DELETE FROM libros WHERE anio_publicacion < 1500;
```

Ejercicio 10. Relaciones

```
CREATE TABLE editoriales (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100),
    pais VARCHAR(50)
);
```

```
ALTER TABLE libros
    ADD COLUMN id_editorial INT,
    ADD CONSTRAINT fk_editorial FOREIGN KEY (id_editorial) REFERENCES editoriales (id)
    ON DELETE CASCADE
    ON UPDATE CASCADE;
```

```
INSERT INTO editoriales (nombre, pais) VALUES
```

```

("Calevo", "Argentina"),
("Lorez", "España"),
("Fully", "EEUU");

UPDATE libros SET id_editorial = 1 WHERE id % 2;
UPDATE libros SET id_editorial = 2 WHERE NOT id % 2;
UPDATE libros SET id_editorial = 3 WHERE id = 3;

SELECT l.titulo AS libro, e.nombre AS editorial, e.pais FROM libros l, editoriales e WHERE
l.id_editorial = e.id;

```

Reto Final. Videoclub.

```

DROP DATABASE IF EXISTS videoclub;
CREATE DATABASE IF NOT EXISTS videoclub;
USE videoclub;

CREATE TABLE peliculas (
    id INT PRIMARY KEY AUTO_INCREMENT,
    titulo VARCHAR(100),
    director VARCHAR(100),
    anio INT,
    precio_alquiler FLOAT(2)
);

CREATE TABLE clientes (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100),
    email VARCHAR(100) NOT NULL,
    CHECK (email LIKE "%@%.%")
);

INSERT INTO peliculas (titulo, director, anio, precio_alquiler) VALUES
("Matrix", "Lana y Lilly Wachowski", 1999, 3.99),
("Gladiator", "Ridley Scott", 2000, 4.50),
("Interstellar", "Christopher Nolan", 2014, 4.99),
("Parásitos", "Bong Joon-ho", 2019, 3.50),
("Pulp Fiction", "Quentin Tarantino", 1994, 2.99);

INSERT INTO clientes (nombre, email) VALUES
("Juan Pérez", "juan.perez@gmail.com"),
("María García", "maria.garcia@hotmail.com"),
("Carlos López", "carlos.lopez@yahoo.es"),
("Laura Sánchez", "laura.sanchez@gmail.com");

SELECT * FROM peliculas;
SELECT * FROM peliculas WHERE anio > 2015;
SELECT * FROM peliculas WHERE precio_alquiler < 4;
SELECT * FROM peliculas WHERE titulo LIKE "%Pulp%";
UPDATE peliculas SET precio_alquiler = 1.99 WHERE titulo LIKE "Ma%";
DELETE FROM peliculas WHERE titulo = "Parásitos";

```

Yo creo que las dos tablas necesitan persistencia, pero más la de clientes que la de películas. Pues son datos necesarios e imprescindibles de la empresa. Si no se persiste eso que es lo más importante ¿qué se persistiría entonces?

Java es lo que ve el usuario final, en lo que se programa y con lo que interactúa el cliente final. Esto se comunica con un SGBD gracias a la API de bajo nivel JDBC, y dicho SGBD está relacionado con la base de datos en sí.